# Crafting Network Packets with Scapy and Traffic Analysis Using Wireshark

Computer Systems Engineer Juan Pablo Vega Villamil

***Abstract*—This laboratory report explores the creation and manipulation of network packets using Scapy, a Python-based packet manipulation framework. The objective of the exercise is to perform reconnaissance on a target system through the creation of custom ICMP and TCP packets. Additionally, network traffic capture and analysis techniques using tcpdump and Wireshark are demonstrated. The report explains the structure of IPv4 headers, packet sniffing techniques, and how attackers can analyze captured traffic to extract sensitive information such as credentials and session cookies.**

***Index Terms*—Scapy, Packet Crafting, Wireshark, Tcpdump, Network Security, Packet Sniffing, ICMP, TCP SYN Scan.**

## I. Introduction

Penetration testers and ethical hackers frequently rely on specially crafted packets to identify vulnerabilities within systems and network infrastructures. These packets allow testers to probe hosts, detect open ports, and analyze network behavior.

Scapy is a powerful packet manipulation program written in Python that allows users to create, send, capture, and analyze packets at multiple protocol layers. According to its author:

> ”Scapy is a Python program that enables the user to send, sniff, dissect and forge network packets. This capability allows construction of tools that can probe, scan or attack networks.”

Unlike traditional tools that focus on specific tasks, Scapy integrates multiple capabilities into a single framework. It can perform tasks such as packet sniffing, tracerouting, scanning, and even attack simulation.

## II. Laboratory Objectives

The main objectives of this laboratory are:

- Use Scapy to construct and send custom network packets.
- Perform reconnaissance on a target host using ICMP and TCP packets.
- Capture network traffic using tcpdump.
- Analyze captured traffic using Wireshark.
- Identify sensitive information such as credentials and session cookies in network traffic.

## III. Accessing Scapy and Exploring Protocols

Scapy is executed from the Linux terminal with root privileges:

```
sudo su
scapy
```

Once inside the Scapy environment, the command `ls()` allows the user to list supported protocols and packet structures.

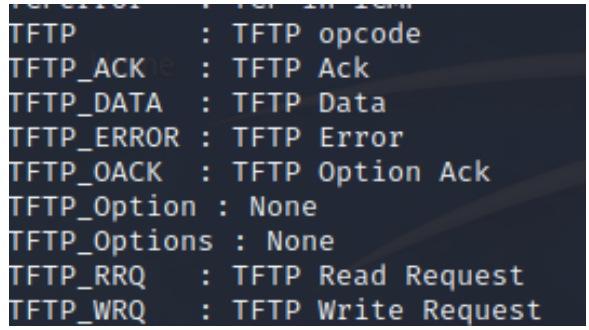


Fig. 1. Listing available protocol layers in Scapy.

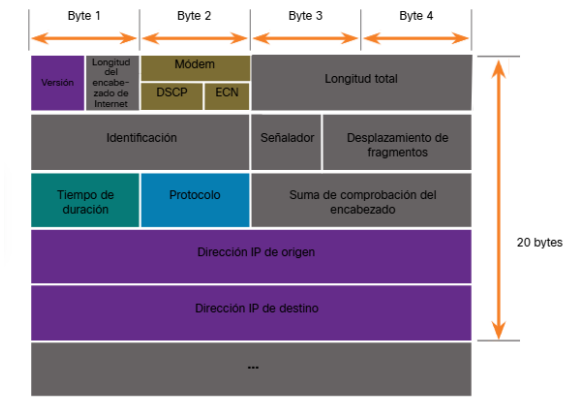


Fig. 2. Structure of an IPv4 packet header.

Among the listed protocols is the Trivial File Transfer Protocol (TFTP). TFTP is commonly used within local area networks to transfer configuration files between network devices.

Important TFTP packet types include:

- TFTP: Base TFTP opcode structure
- TFTP_ACK: Acknowledgment packet
- TFTP_DATA: Packet carrying file data
- TFTP_ERROR: Error notification packet
- TFTP_OACK: Option acknowledgment
- TFTP_RRQ: Read request packet
- TFTP_WRQ: Write request packet

## IV. Examining IPv4 Packet Headers

The IPv4 header contains several fields that control how packets travel through the network.

Key fields include:

**Version:** A 4-bit value identifying the IP version. IPv4 packets contain the binary value 0100.

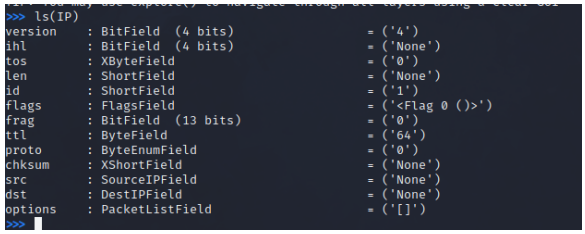


Fig. 3. Displaying IPv4 fields in Scapy.

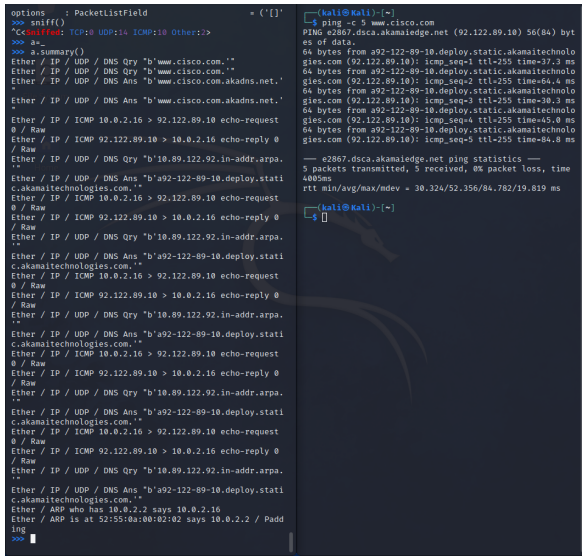

Fig. 4. Packet sniffing using Scapy.

**Differentiated Services (DS):** An 8-bit field used to determine packet priority and quality of service. The first six bits correspond to DSCP while the last two represent Explicit Congestion Notification (ECN).

**Time to Live (TTL):** An 8-bit value that limits the lifetime of the packet. Each router decrements the TTL value by one. When it reaches zero, the packet is discarded and an ICMP time-exceeded message is returned.

**Protocol:** Identifies the transport layer protocol contained in the packet payload. Common values include ICMP (1), TCP (6), and UDP (17).

**Header Checksum:** Used to detect corruption in the IPv4 header.

**Source Address:** 32-bit address representing the sender.

**Destination Address:** 32-bit address identifying the receiver.

## V. Enumerating IPv4 Fields with Scapy

The command `ls(IP)` displays all fields contained within an IPv4 header.

This functionality allows users to understand how packets are structured before modifying or creating them.

## VI. Packet Sniffing with Scapy

Scapy can capture live network traffic using the `sniff()` function.

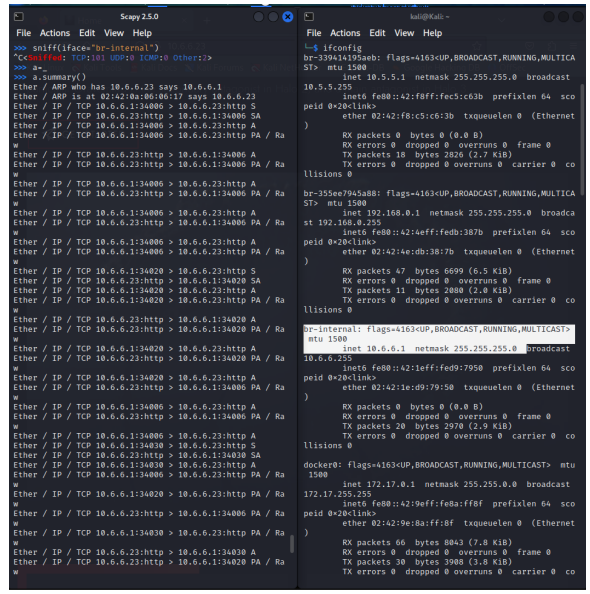

Fig. 5. Summary of captured network packets.

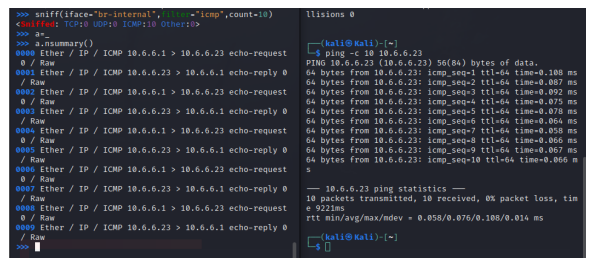

Fig. 6. Captured ICMP echo requests and replies.

Captured packets can then be summarized using the `summary()` function.

The sniffing process can be limited to a specific network interface:

```
sniff(iface="br-internal")
```

When performing a ping to host 10.6.6.23, ICMP echo request and echo reply packets are captured.

Examining packet details reveals multiple protocol layers including Ethernet, IP, and ICMP. Two sets of source and destination addresses appear because one corresponds to MAC addresses and the other to IP addresses.

Captured packets can be stored using:

```
wrpcap("capture1.pcap", a)
```

## VII. Creating and Sending ICMP Packets

Scapy allows the creation of custom ICMP packets.

```
send(IP(dst="10.6.6.23")/ICMP()/"This is a test")
```

The packet payload now contains the custom message.

## VIII. Creating and Sending TCP SYN Packets

TCP SYN packets can also be crafted:

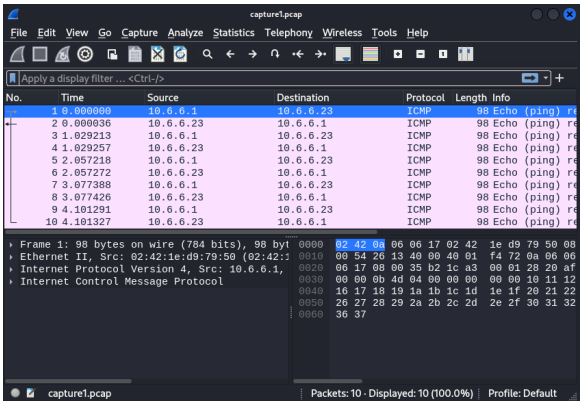


Fig. 7. Opening the capture file in Wireshark.

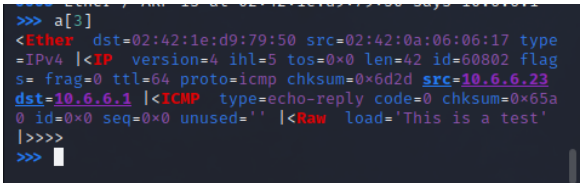


Fig. 8. Sending a custom ICMP packet.

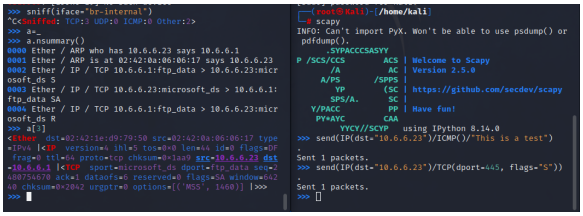


Fig. 9. Sending a TCP SYN packet.

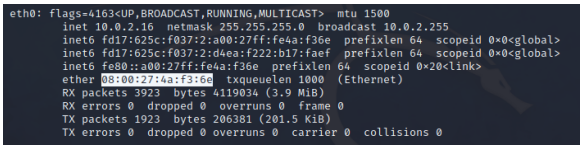


Fig. 10. Identifying the network interface and MAC address.

```
send(IP(dst="10.6.6.23")/TCP(dport=445,flags="S"))
```

If the destination port is open, the target system responds with a SYN-ACK packet. The SA flag indicates that the port is accepting connections.

By sending multiple SYN packets to different ports, attackers can determine which services are running on a host.

## IX. Network Traffic Capture with Tcpdump

Tcpdump is a command-line packet capture tool widely used in Linux systems.

First, the network interface and MAC address must be identified.

Traffic capture is performed using:

```
sudo tcpdump -i eth0 -s 0 -w packetdump.pcap
```

Where:

- **-i** specifies the interface.
- **-s 0** captures the full packet.
- **-w** writes the output to a file.

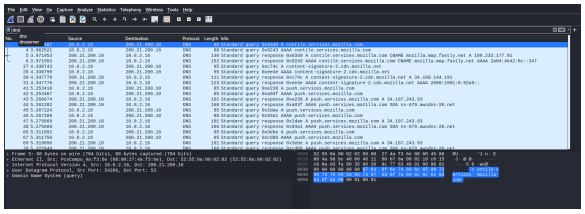

Fig. 11. DNS queries captured in Wireshark.

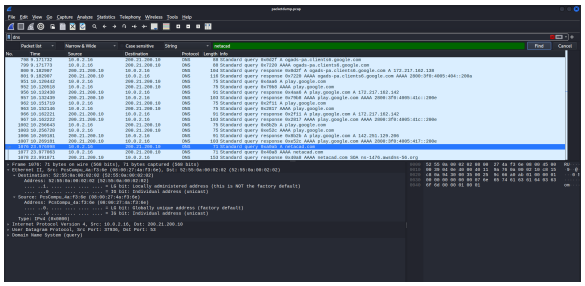

Fig. 12. Filtering DNS traffic for NetAcad.

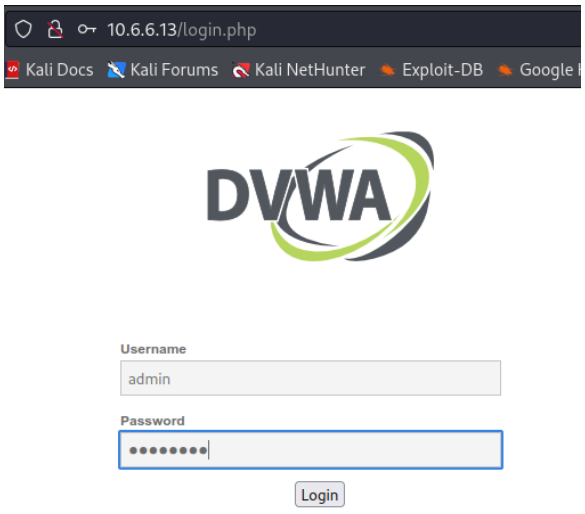


Fig. 13. Login request to the vulnerable host.

## X. Traffic Analysis with Wireshark

The generated capture file can be analyzed in Wireshark.

Applying a DNS filter reveals queries to websites such as Google, Mozilla, and NetAcad.

Filtering by string values allows identification of specific domains.

## XI. Credential Capture in HTTP Traffic

In an insecure network, login credentials may be transmitted in plaintext.

A login request is performed to a vulnerable host.

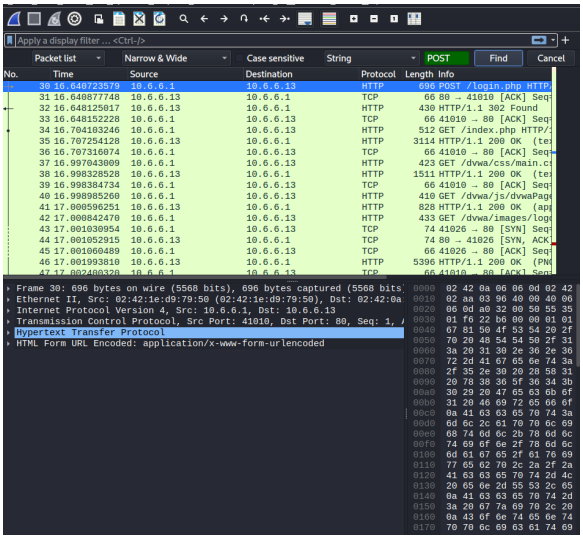


Fig. 14. Captured HTTP POST request.

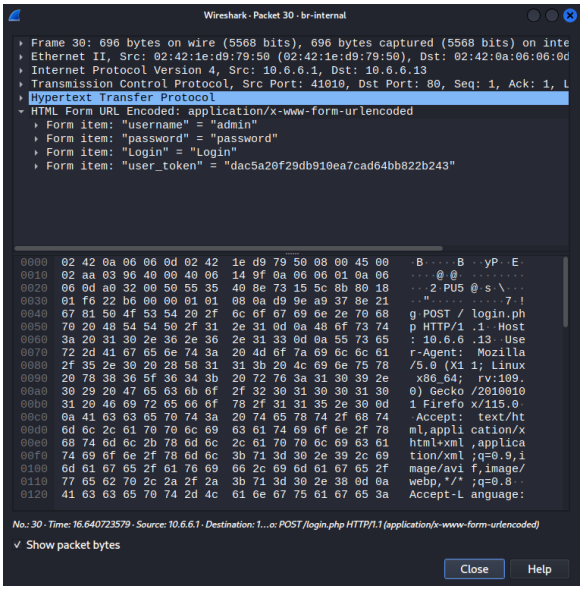


Fig. 15. Credentials visible in captured packet data.

Filtering HTTP POST packets reveals transmitted credentials.

Opening the packet details exposes the login credentials.

Session cookies are also visible in HTTP responses. These cookies store session identifiers such as PHPSESSID. If intercepted, attackers could potentially hijack user sessions.

## XII. Conclusion

This laboratory demonstrated the capabilities of Scapy for packet crafting and network reconnaissance. Through the creation of custom ICMP and TCP packets, it was possible to analyze host behavior and detect open ports.

Additionally, tcpdump and Wireshark were used to capture and analyze network traffic. The analysis revealed how sensitive information such as credentials and session cookies can be exposed when transmitted over insecure protocols.

These techniques highlight the importance of encryption and secure communication protocols in modern network infrastructures.